LAYER 7 · APPLICATION

L7

Apps, APIs and the Services Underneath

What people actually use: React and Next.js on the front, NestJS and Node.js behind, REST and HTTP between them, and the DNS and DHCP that make the rest possible.

YASIR ABDULLAH · YASIR-PORTFOLIO-ZETA.VERCEL.APP/NOTES/EBOOKS/LAYER-7-APPLICATION

SKIP TO CONTENT

Contents

1 **React**

Building interfaces from components: data flows down as props, events flow back up.

2 **Next.js**

React with a server: rendering on the server, streaming HTML, and small client islands.

3 **NestJS**

A structured Node.js backend: modules, controllers, services and the request pipeline.

4 **Node.js**

JavaScript on the server, and how one thread serves thousands of requests with the event loop.

5 **REST APIs**

Resources, methods and status codes: designing an API that behaves the way people expect.

6 **HTTP and HTTPS**

Requests and responses, methods, headers and status codes, and what the S adds.

7 **DNS**

Turning names into addresses: resolvers, the hierarchy, record types and caching.

SKIP TO CONTENT

How a device gets its address, mask, gateway and DNS without anyone typing them.

SKIP TO CONTENT

CHAPTER 1 · REACT

React

Building interfaces from components: data flows down as props, events flow back up.



State lives in the parent. It flows down as props; a click in a child calls a function from the parent, which updates state and re-renders.

React builds user interfaces out of **components**: functions that take some data and return what the screen should look like. When the data changes, React works out what changed on screen and updates only that.

Components and props

TSX

SKIP TO CONTENT

```
id: number; name: string; price: number };

function ProductCard({ product, onAdd }: { product: Product; onAdd: (p:
```

```
  return (
    <article>
      <h3>{product.name}</h3>
      <p>₹{product.price}</p>
      <button onClick={() => onAdd(product)}>Add</button>
    </article>
  );
}
```

Props are the inputs. They flow **down**, from parent to child, and a child never changes its own props.

State

State is data a component owns and can change. Changing it with the setter re-renders that component and its children.

TSX

```tsx
function Cart({ products }: { products: Product[] }) {
  const [items, setItems] = useState<Product[]>([]);
  const total = items.reduce((sum, item) => sum + item.price, 0);

  return (
    <>
      <ProductList products={products} onAdd={(p) => setItems([...items,
      <Total amount={total} />
    </>
  );
}
```

Notice that `total` is not state. It is **derived** from `items` during render, so it can never fall out of sync.

Data down, events up

The diagram shows the pattern that keeps React apps understandable:

SKIP TO CONTENT

closest common parent, here `Cart`.

2. It flows down as props to `ProductList` and `Total`.

3. A click in `ProductCard` calls a function passed down from `Cart`.
4. `Cart` updates its state, and everything that depends on it re-renders.

Effects are for the outside world

`useEffect` synchronises a component with something outside React, such as a subscription, a timer or a browser API. It is not for computing values from props or state; do that during render.

```tsx
useEffect(() => {
  const id = setInterval(refresh, 30_000);
  return () => clearInterval(id);
}, [refresh]);
```

Keys in lists

```tsx
<ul>
  {products.map((product) => (
    <ProductCard key={product.id} product={product} onAdd={add} />
  ))}
</ul>
```

A stable `key` lets React match each item between renders. Using the array index breaks when items are added, removed or reordered.

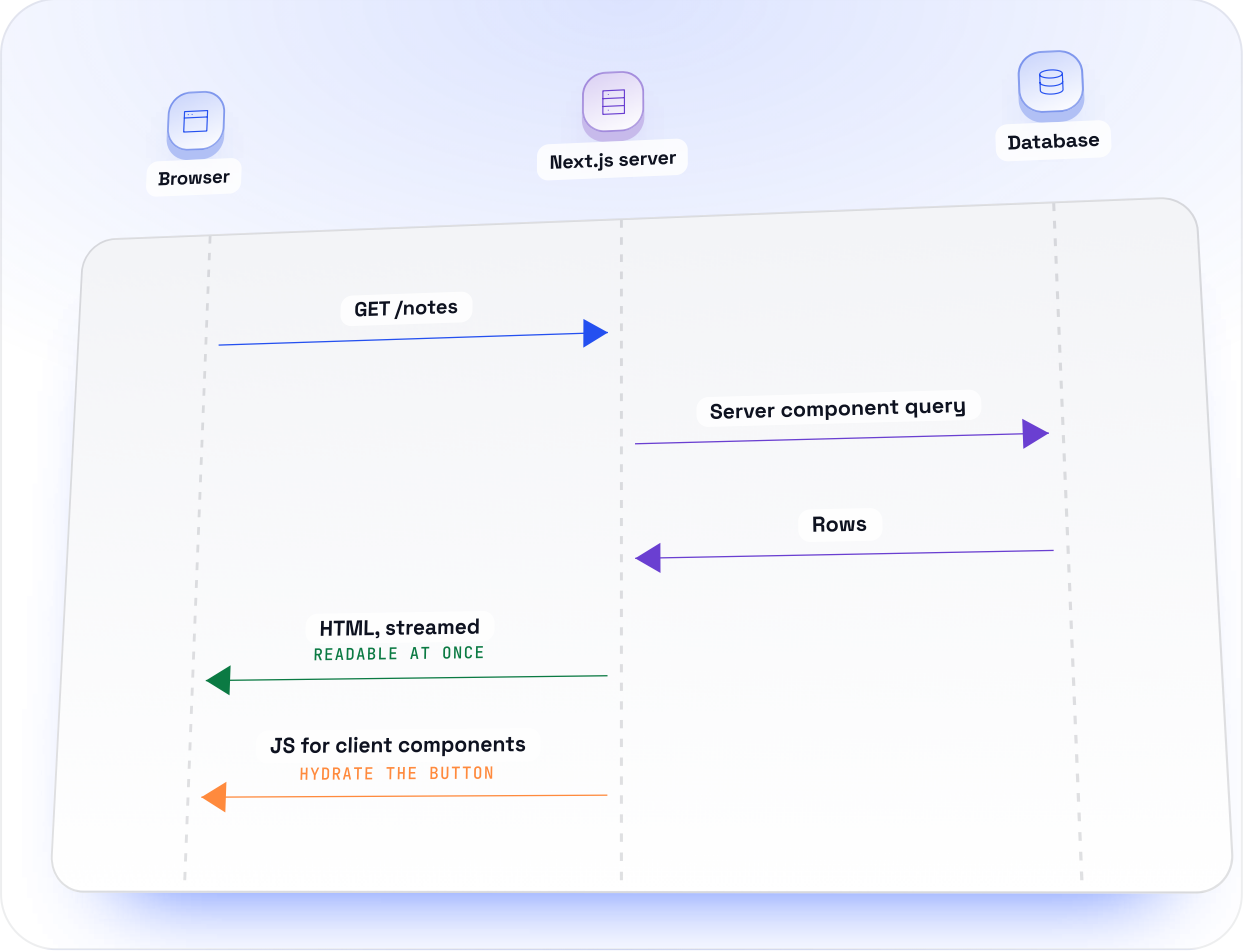
When it goes wrong

- **The screen does not update:** state was mutated (`items.push(p)`) instead of replaced with a new array.
- **"Too many re-renders":** a setter is called during render instead of inside an event handler.
- **s while typing:** a component is defined inside another is recreated on every render.

SKIP TO CONTENT

Next.js

React with a server: rendering on the server, streaming HTML, and small client islands.



The server renders the page with its data and streams HTML. The browser shows it at once, then hydrates only the interactive parts.

Next.js is a framework for building React apps with a server. Pages are rendered on the server, with their data, and sent as HTML that appears immediately. Only the parts that need to be interactive ship JavaScript to the browser.

Routing from folders

In the App Router, folders under `app/` are URLs:

SKIP TO CONTENT

```
app/
  layout.tsx            shared shell for every page
  page.tsx              /
  notes/
    page.tsx            /notes
    [slug]/
      page.tsx          /notes/subnetting-without-the-headache
```

A `[slug]` folder is a dynamic segment; its value arrives in `params`.

Server components by default

Components in `app/` run on the server unless marked otherwise. They can read the database directly and never send that code to the browser.

TSX

```tsx
export default async function NotesPage() {
  const notes = await db.note.findMany({ where: { published: true } });
  return (
    <ul>
      {notes.map((note) => (
        <li key={note.slug}>{note.title}</li>
      ))}
    </ul>
  );
}
```

Client components for interaction

Anything that needs state, effects or browser events gets `"use client"` at the top of its file:

TSX

```tsx
"use client";

LikeButton() {
  setLiked] = useState(false);
```

SKIP TO CONTENT

```
  return <button onClick={() => setLiked(!liked)}>{liked ? "Liked" : "Li
}
```

A server component can render a client component, passing serialisable props. Keep client components small and near the leaves, so most of the page stays server-rendered.

Streaming

With `<Suspense>`, the server sends the page shell at once and streams slower sections in as their data arrives. The user reads the top of the page while the rest loads.

Changing data

Server actions let a form call a server function directly:

```tsx
async function save(formData: FormData) {
  "use server";
  // validate, write to the database, then revalidate the cached page
}

<form action={save}>…</form>;
```

Caching and rendering

Pages whose data does not change per request can be prerendered at build time and served instantly; pages that depend on the request are rendered on demand. Each major version has refined how this is configured, so check the docs for the version you run.

When it goes wrong

SKIP TO CONTENT

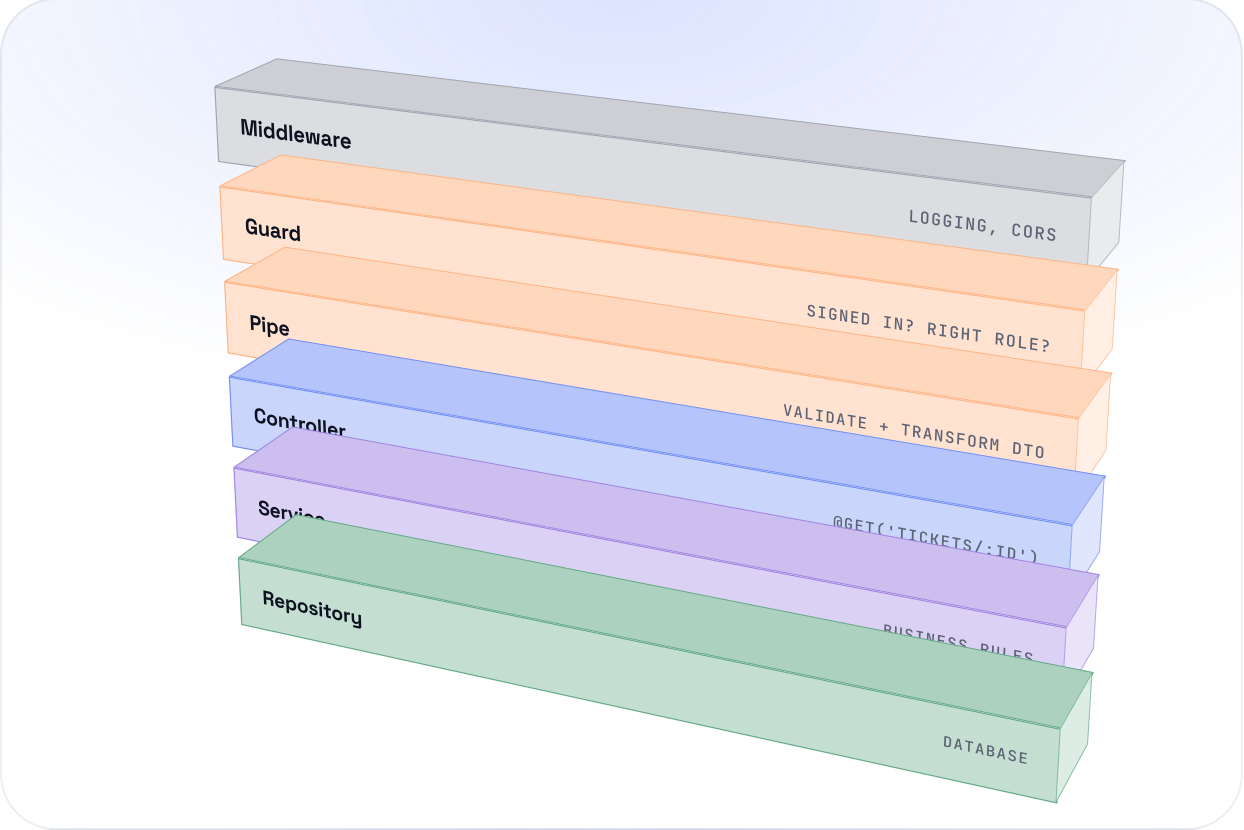
”: the server and browser rendered different HTML, often from dates, random numbers or `window` checks during render.

- **”useState only works in client components”:** add `"use client"` to that file, or move the interactive part into a small client component.
- **A secret appears in the browser bundle:** only variables prefixed `NEXT_PUBLIC_` should ever reach client code.

SKIP TO CONTENT

NestJS

A structured Node.js backend: modules, controllers, services and the request pipeline.



A request passes through the pipeline in a fixed order. Anything that fails a guard or a pipe never reaches your business logic.

NestJS is a Node.js framework for building backends with a clear structure. It borrows ideas from Angular and Spring: **modules** group features, **controllers** handle HTTP, **services** hold the business logic, and dependency injection wires them together.

The building blocks

TS

```ts
@Controller("tickets")
cketsController {
ivate readonly tickets: TicketsService) {}
```

SKIP TO CONTENT

```
  @Get(":id")
  findOne(@Param("id", ParseIntPipe) id: number) {
    return this.tickets.findOne(id);
  }

  @Post()
  @UseGuards(AuthGuard, RolesGuard)
  @Roles("staff")
  create(@Body() dto: CreateTicketDto) {
    return this.tickets.create(dto);
  }
}
```

TS

```
@Injectable()
export class TicketsService {
  constructor(private readonly prisma: PrismaService) {}

  async findOne(id: number) {
    const ticket = await this.prisma.ticket.findUnique({ where: { id } });
    if (!ticket) throw new NotFoundException();
    return ticket;
  }
}
```

The controller stays thin: it reads the request and calls the service. The service knows nothing about HTTP, which makes it easy to test.

The request pipeline

Every request passes through the same stages, in order:

1. **Middleware:** logging, CORS, request IDs.
2. **Guards:** may this request continue? Authentication and roles live here.
3. **Interceptors (before):** timing, caching, transforming.
4. **Pipes:** validate and convert input, for example the DTO and `ParseIntPipe`.
5. **rvice:** your code.
6. **nterceptors (after)** and **exception filters:** shape the response or the error.

SKIP TO CONTENT

If a guard or pipe rejects the request, your business logic never runs.

Validating input with DTOs

```ts
export class CreateTicketDto {
  @IsString()
  @Length(3, 120)
  title: string;

  @IsIn(["low", "normal", "high"])
  priority: string;
}
```

```ts
app.useGlobalPipes(new ValidationPipe({ whitelist: true, forbidNonWhitel
```

`whitelist` strips unknown fields, so a client cannot set fields you did not declare.

Modules

```ts
@Module({
  imports: [PrismaModule],
  controllers: [TicketsController],
  providers: [TicketsService],
})
export class TicketsModule {}
```

One module per feature keeps a large codebase navigable: tickets, users, billing.

When it goes wrong

SKIP TO CONTENT

ve dependencies of TicketsService”: the provider it needs is not in this module or not exported from the module that owns it.

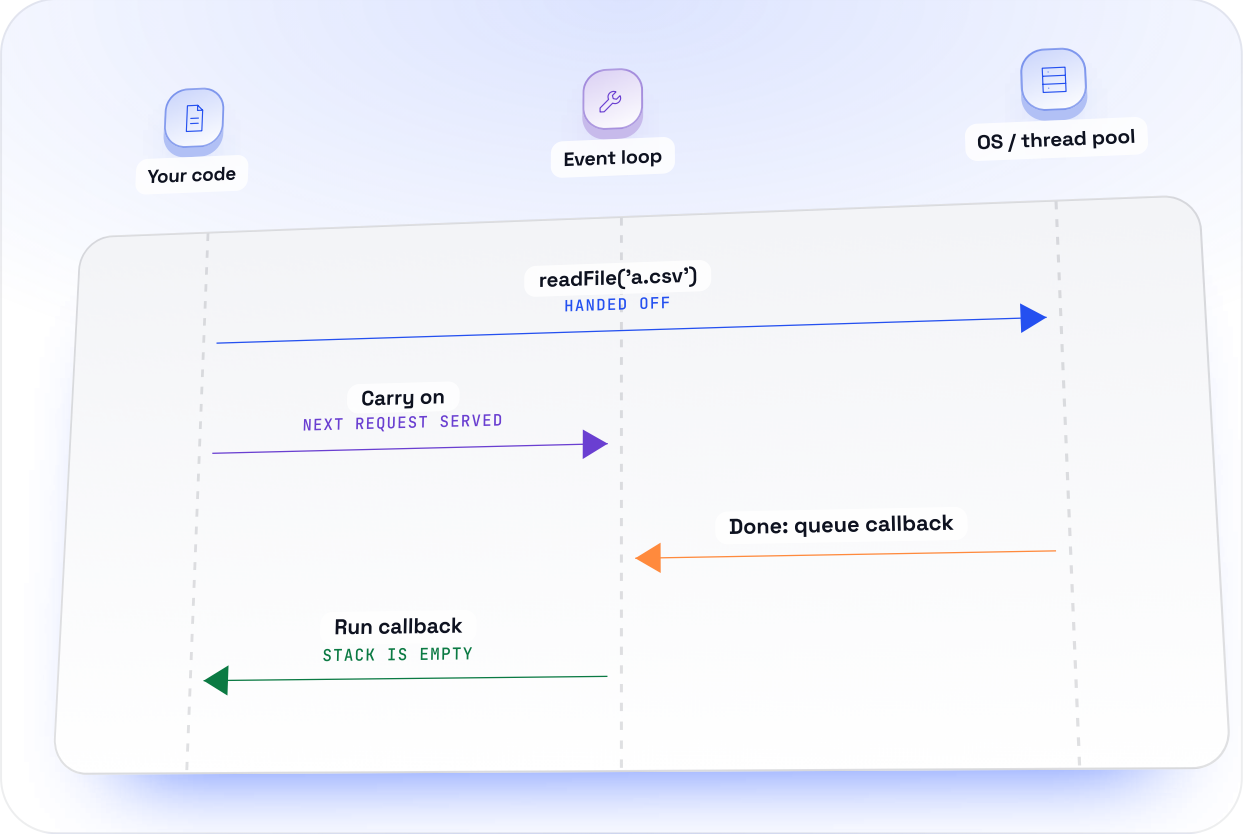
- **Validation never runs:** the global `ValidationPipe` is missing, or the DTO is an interface instead of a class.
- **A guard seems ignored:** it is applied to the wrong controller or method, or the route is marked public.

SKIP TO CONTENT

CHAPTER 4 · NODE.JS

Node.js

JavaScript on the server, and how one thread serves thousands of requests with the event loop.



Slow work is handed off. Your code keeps going, and when the operating system finishes, the callback is queued and the event loop runs it.

Node.js runs JavaScript outside the browser, on servers and command lines. Your JavaScript runs on **one thread**, yet a single Node process can serve thousands of connections. The reason is the **event loop**: slow work is handed off, and your code is only run when there is something ready to do.

Non-blocking I/O

```ts
import { readFile } from "node:fs/promises";

const data = await readFile("report.csv", "utf8");
```

SKIP TO CONTENT

While the file is being read, Node does not sit and wait. It hands the job to the operating system (or its own small thread pool), and the thread goes on serving other requests. When the read finishes, the callback is queued, and the event loop runs it as soon as the call stack is empty.

Don’t block the loop

Because there is one JavaScript thread, anything that keeps it busy stalls **every** user:

```ts
// Blocks the whole server while it runs
const hash = crypto.pbkdf2Sync(password, salt, 600_000, 32, "sha256");

// Hands the work off; other requests keep flowing
const hash = await promisify(crypto.pbkdf2)(password, salt, 600_000, 32,
```

- Prefer the async versions of file, crypto and compression functions.
- Move heavy CPU work, such as large image processing, to worker threads or a separate service.
- Avoid huge synchronous loops over big arrays in request handlers.

Modules and packages

```json
{
  "type": "module",
  "scripts": { "start": "node server.js", "dev": "node --watch server.js
  "engines": { "node": "⩾22" }
}
```

- `npm install` adds packages; commit `package-lock.json` so every machine installs the same versions.
- `npm ci` installs exactly from the lockfile; use it in CI and production builds.
- environment variables, read with `process.env.NAME`, never in

SKIP TO CONTENT

A minimal server

```ts
import { createServer } from "node:http";

createServer((req, res) => {
  res.writeHead(200, { "Content-Type": "application/json" });
  res.end(JSON.stringify({ ok: true }));
}).listen(3000);
```

Frameworks such as Express, Fastify and NestJS build on this.

Errors that crash the process

An exception that nobody catches, or a rejected promise that nobody handles, can stop the process. Always `await` promises inside `try/catch` where failure is possible, and run production servers under a supervisor (systemd, PM2, a container platform) that restarts them.

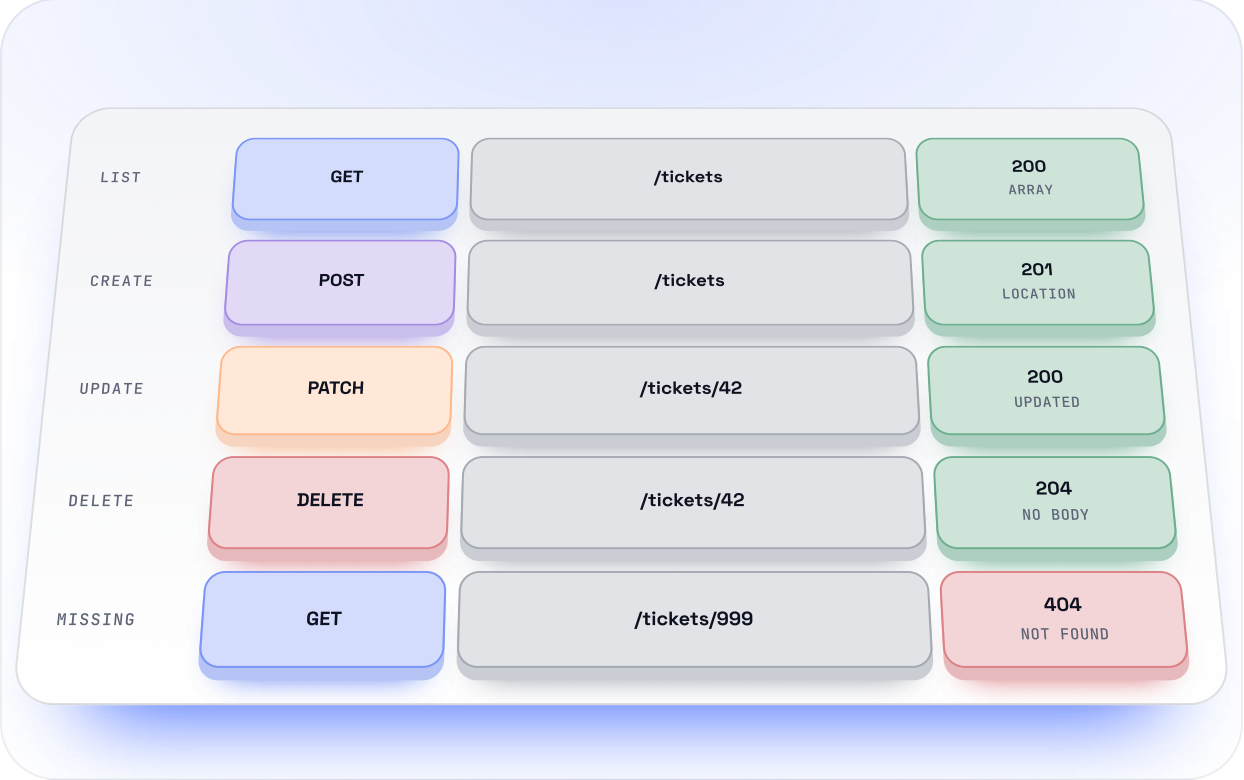
When it goes wrong

- **The whole API freezes for seconds at a time:** something synchronous and heavy is blocking the loop.
- **Memory climbs until the process dies:** an ever-growing cache, array or listener list.
- **Works locally, not on the server:** different Node version. Pin it in `engines` and in the deployment settings.

SKIP TO CONTENT

REST APIs

Resources, methods and status codes: designing an API that behaves the way people expect.



Nouns in the URL, verbs in the method, and a status code that tells the client what happened.

REST is a style for designing web APIs around **resources**. Each resource has a URL, the HTTP method says what to do with it, and the status code says what happened. When an API follows these conventions, developers can guess how it works before reading the docs.

Nouns in the URL, verbs in the method

METHOD	URL	MEANING	SUCCESS
GET	/tickets	List tickets	200
	kets	Create a ticket	201

SKIP TO CONTENT

METHOD	URL	MEANING	SUCCESS
GET	/tickets/42	Read ticket 42	200
PATCH	/tickets/42	Change some fields	200
PUT	/tickets/42	Replace the whole ticket	200
DELETE	/tickets/42	Delete it	204

Avoid verbs in paths such as `/getTickets` or `/deleteTicket?id=42`. Nested resources show relationships: `/tickets/42/comments`.

Status codes that matter

CODE	MEANING
200	OK
201	Created; include a `Location` header
204	Done, no body
400	The request is malformed or fails validation
401	Not authenticated
403	Authenticated, but not allowed
404	No such resource (or not yours to see)
409	Conflict, such as a duplicate
422	Understood, but the data is invalid
429	Too many requests; slow down
500	Server error: a bug, not the client's fault

SKIP TO CONTENT

Safe and idempotent

- **GET** must never change anything. Crawlers and prefetchers will call it.
- **PUT** and **DELETE** are idempotent: sending the same request twice leaves the same result.
- **POST** is not. If a client might retry, accept an `Idempotency-Key` header so a double click does not create two orders.

Consistent responses

JSON

```
{
  "data": [{ "id": 42, "title": "VPN down", "status": "open" }],
  "page": { "next": "/tickets?cursor=eyJpZCI6NDJ9" }
}
```

JSON

```
{
  "error": {
    "code": "validation_failed",
    "message": "Title is required",
    "fields": { "title": "Required" }
  }
}
```

- Paginate lists; never return an unbounded table.
- Filter and sort with query parameters: `/tickets?status=open&sort=-createdAt`.
- Version the API (`/v1/…`) before the first breaking change, not after.

Testing from the command line

SHELL

```
curl -s https://api.example.com/v1/tickets/42 -H "Authorization: Bearer
curl -s -X POST https://api.example.com/v1/tickets \
  -H "Content-Type: application/json" \
  -d '{"title":"Printer offline","priority":"normal"}'
```

SKIP TO CONTENT

When it goes wrong

- **Every error is a 200 with `"success": false`:** clients and monitoring cannot tell failures apart. Use real status codes.
- **CORS errors in the browser only:** the API must send `Access-Control-Allow-Origin` for the site calling it.

SKIP TO CONTENT

CHAPTER 6 · HTTP/HTTPS

HTTP and HTTPS

Requests and responses, methods, headers and status codes, and what the S adds.



HTTPS is HTTP inside TLS. After the handshake, the same request and response travel encrypted.

HTTP is the language browsers and servers speak: the client sends a **request**, the server sends back a **response**. HTTPS is the same conversation carried inside TLS, so nobody in between can read or change it.

A request

```
TEXT
SKIP TO CONTENT
                P/1.1
           com
Accept: text/html
```

```
Cookie: sid=Hk3…
User-Agent: Mozilla/5.0 …
```

The first line has the **method**, the **path** and the version. Headers follow. A POST or PUT adds a blank line and a **body**.

A response

```text
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
Cache-Control: max-age=600
Set-Cookie: theme=dark; Path=/

<!doctype html>…
```

The status line, headers, a blank line, then the body.

Status code families

RANGE	MEANING	EXAMPLES
1xx	Informational	101 Switching Protocols
2xx	Success	200 OK, 201 Created, 204 No Content
3xx	Redirect	301 Moved Permanently, 304 Not Modified
4xx	Client error	400, 401, 403, 404, 429
5xx	Server error	500, 502 Bad Gateway, 503, 504

502 and 504 usually mean a proxy or load balancer could not get an answer from the app behind it. The app, not the proxy, is where to look.

SKIP TO CONTENT

h knowing

- `Cache-Control`: how long browsers and CDNs may reuse a response.

- `Location`: where a redirect or a new resource points.
- `Content-Type`: what the body is, and its charset.
- `Authorization`: credentials such as a bearer token.
- `Strict-Transport-Security`, `Content-Security-Policy`: security policies for the browser.

What the S adds

Before any HTTP is sent, HTTPS performs a TLS handshake. From then on every request and response, including the path, headers and cookies, is encrypted. Only the server's address and, usually, its name are visible to the network.

HTTP/1.1, 2 and 3

- **HTTP/1.1:** one request at a time per connection; browsers open several connections.
- **HTTP/2:** many requests multiplexed on one TCP connection, with compressed headers.
- **HTTP/3:** runs over QUIC on UDP, so one lost packet no longer stalls every request.

The requests and responses look the same to your application in all three.

Inspecting traffic

```shell
curl -I https://example.com
curl -v https://example.com/api/health
```

In the browser, the **Network** tab in DevTools shows every request, its status, headers, timing and body.

When it goes wrong

SKIP TO CONTENT

…e app redirects to HTTPS, and a proxy in front forwards as HTTP, so the app redirects again. Trust the proxy's `X-Forwarded-`

`Proto` header.

- **Old version after a deploy:** a cache is still serving it. Check `Cache-Control` and the CDN.
- **Mixed content warnings:** an HTTPS page loads a script or image over HTTP. Load everything over HTTPS.

SKIP TO CONTENT

DNS

Turning names into addresses: resolvers, the hierarchy, record types and caching.



The resolver asks down the hierarchy, root to .com to the domain's own server, then caches the answer for its TTL.

DNS (the Domain Name System) turns names that people remember, such as into the IP addresses that computers need. Almost work depends on it, which is why "the internet is down" so often turns out to be DNS.

SKIP TO CONTENT

How a name is resolved

1. The laptop asks its configured **resolver** (the router, the ISP, or a public one such as 1.1.1.1 or 8.8.8.8).
2. If the resolver has a cached answer, it replies at once.
3. Otherwise it asks a **root** server, which points it to the servers for `.com`.
4. The `.com` servers point it to the **authoritative** name servers for `example.com`.
5. Those servers give the actual answer.
6. The resolver caches the answer for its **TTL** and returns it to the laptop, which caches it too.

Record types

TYPE	HOLDS	EXAMPLE
A	IPv4 address	`example.com ⟶ 93.184.216.34`
AAAA	IPv6 address	`example.com ⟶ 2606:2800:…`
CNAME	Alias to another name	`www ⟶ example.com`
MX	Mail servers, with priority	`10 mail.example.com`
TXT	Text: SPF, DKIM, domain verification	`v=spf1 include:_spf.google.co`
NS	The domain's authoritative servers	`ns1.provider.net`
PTR	Reverse lookup: IP to name	`34.216.184.93.in-addr.arpa ⟶`

Asking DNS yourself

WINDOWS CMD

```
nslookup example.com
nslookup -type=mx example.com 8.8.8.8
ipconfig /displaydns
```

SKIP TO CONTENT

```shell
dig example.com
dig +short MX example.com
dig @1.1.1.1 example.com
```

Asking a specific server (`8.8.8.8`, `1.1.1.1`) tells you whether the problem is your resolver or the record itself.

TTL and changes

A record's TTL says how long resolvers may cache it. Before moving a site to a new server, lower the TTL (say to 300 seconds) a day ahead. After the move, raise it again.

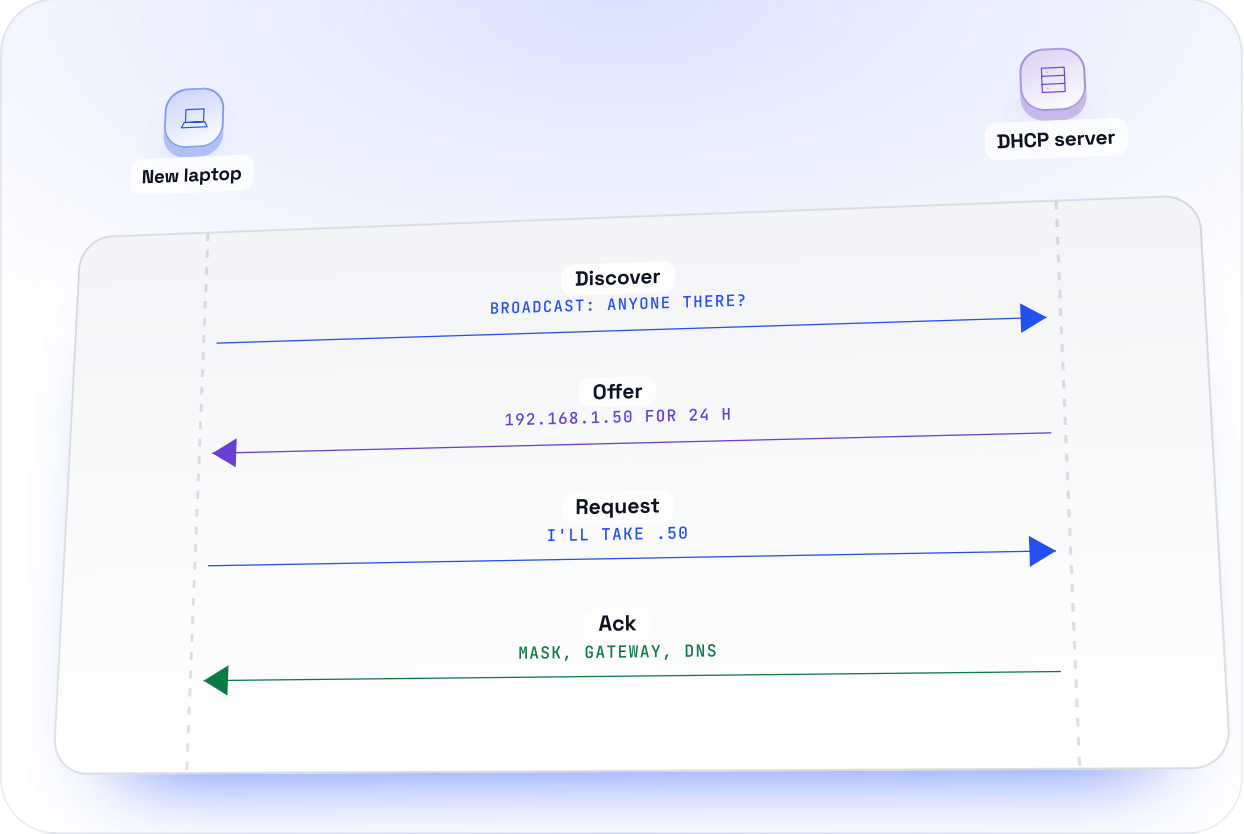
When it goes wrong

SYMPTOM	LIKELY CAUSE
`ping 8.8.8.8` works, `ping google.com` fails	The PC's DNS server is wrong or un
New site works for some people only	Old answer still cached; wait out t
One internal name fails	Missing record on the internal DNS
Email not arriving	Missing or wrong MX, or SPF/DKIM
Everything slow to start, then fine	A dead DNS server listed first; the

SKIP TO CONTENT

DHCP

How a device gets its address, mask, gateway and DNS without anyone typing them.



Discover, Offer, Request, Acknowledge. Four messages and the laptop has an address, a gateway and DNS servers for the lease time.

DHCP (Dynamic Host Configuration Protocol) is why you can plug a laptop into any office port or join any Wi-Fi and simply be online. A server hands out an IP address and the other settings a device needs, for a limited time called the **lease**.

DORA: four messages

1. **Discover:** the new device has no address, so it broadcasts: "is there a DHCP server?"
2. **Off** replies with an address it can lend, such as 192.168.1.50 for

SKIP TO CONTENT

3. **Request:** the device asks for that address (broadcast, so other servers know their offers were declined).
4. **Acknowledge:** the server confirms and sends the full settings.

Halfway through the lease, the device quietly asks the same server to renew it.

What the server hands out

- IP address and subnet mask
- Default gateway
- DNS servers
- Lease time
- Optional extras, such as a domain name, NTP server, or the TFTP server for IP phones

A scope on a Cisco router

CISCO IOS

```
ip dhcp excluded-address 192.168.1.1 192.168.1.20
!
ip dhcp pool OFFICE
 network 192.168.1.0 255.255.255.0
 default-router 192.168.1.1
 dns-server 1.1.1.1 8.8.8.8
 lease 1
```

Exclude the addresses you give to routers, printers and servers by hand, so DHCP never lends them out.

CISCO IOS

```
show ip dhcp binding
show ip dhcp pool
show ip dhcp conflict
```

SKIP TO CONTENT

Across VLANs: the relay

Discover is a broadcast, and broadcasts stop at the router. If the DHCP server is on another subnet, the router must forward the requests:

CISCO IOS

```
interface GigabitEthernet0/0.20
 ip helper-address 10.50.0.5
```

Reservations

For a printer that must keep its address, create a reservation on the server tied to its MAC address. It still uses DHCP, but always receives the same address.

On the client

WINDOWS CMD

```
ipconfig /all
ipconfig /release
ipconfig /renew
```

SHELL

```
sudo dhclient -r && sudo dhclient
```

When it goes wrong

SYMPTOM	LIKELY CAUSE
169.254.x.x address	No DHCP reply: cable, VLAN, relay or the server is
Some users get no address	The pool is full; shorten the lease or widen the s
Address conflict warnings	A static device inside the pool
Devices get a strange gateway	A rogue DHCP server, such as a home router plug